Trabajo Práctico 7 — Integración de APIs y SDKs

Cátedra: Arquitecturas Móviles — UTN Facultad Regional San Francisco
Profesor: Ing. Juan Pablo Bono
Alumnos: Martiniano Gimenez, Fabricio Quinteros
Aplicación: Tarjetazo — gastos de una tarjeta de crédito compartida
Repositorio: https://github.com/FabriQuinteros/tp1-arquitecturas-moviles (rama `feat/tp7-apis-sdks`)

---

1. API y SDK: qué se integra y cómo

Una API es el contrato con que un servicio expone sus operaciones, casi siempre como un servicio REST. Un SDK es la biblioteca que implementa ese contrato para una plataforma: se encarga de la conexión, la autenticación, los reintentos y la conversión de datos, y deja al código de la app una llamada de una línea. El costo es una dependencia más que hay que mantener actualizada.

En Tarjetazo se usan las dos formas. Los servicios de Firebase entran como SDK, declarados como dependencias de Gradle bajo una misma lista de versiones (Firebase BoM): se actualiza un solo número y todas las bibliotecas de Firebase quedan compatibles entre sí. La cotización del dólar entra como API REST directa, porque es una sola llamada `GET` que devuelve un JSON y no justifica sumar una biblioteca.

Cada dependencia externa queda detrás de una sola clase (`Repositorio` para Firestore, `Cotizacion` para la API): si mañana se cambia de proveedor, cambia esa clase y no las pantallas.

2. Selección

Integración	Tipo	Qué resuelve en Tarjetazo	Por qué esta y no otra
Cloud Firestore	SDK de Firebase	Los gastos quedaban en el teléfono y no dependían de la cuenta: dos personas en el mismo dispositivo veían lo mismo y quien cambiaba de teléfono los perdía. Ahora se guardan por cuenta en la nube.	Ya se usa Firebase Authentication, así que la identidad del usuario sirve directo para las reglas de seguridad. Trae caché local: la carga sigue funcionando sin conexión y se sincroniza al volver la señal, que era una condición del MVP. Escala sola, sin servidores propios.
Firebase Cloud Messaging	SDK de Firebase	El titular se olvida de cargar los gastos hasta que llega el resumen, que es justo lo que la app quiere evitar. Un aviso antes del cierre de la tarjeta le recuerda revisar y dividir lo pendiente.	Es el único canal para que un servidor le hable al teléfono sin que la app esté abierta. Con temas, quien publica no necesita conocer cada dispositivo: publica una vez y le llega a todos los suscriptos. Gratuito y del mismo proyecto de Firebase.
DolarApi (`dolarapi.com`)	API REST pública	Los consumos en dólares eran un caso que el MVP dejaba afuera. El banco los cobra al "dólar tarjeta", que ya incluye los impuestos; la app convierte el importe a ese valor del día.	Gratuita, sin clave, responde JSON simple y publica justamente el dólar tarjeta. Al no haber SDK, se consume con `HttpURLConnection` y `org.json`, que ya vienen con Android.

3. Implementación

3.1 Dependencias

```
// app/build.gradle.kts
implementation(platform(libs.firebase.bom))   // una versión para todo Firebase
implementation(libs.firebase.auth)            // TP 2
implementation(libs.firebase.firestore)       // nuevo
implementation(libs.firebase.messaging)       // nuevo
```

Y en el manifiesto, el permiso `INTERNET` para la API del dólar, `POST_NOTIFICATIONS` (desde Android 13 mostrar avisos requiere permiso explícito) y el servicio que recibe los mensajes.

3.2 Cloud Firestore — `Repositorio.kt`

El repositorio pasa a tener dos niveles. `SharedPreferences` sigue siendo la copia local que lee la interfaz (ahora una por cuenta) y Firestore guarda cada gasto como un documento en `usuarios/{uid}/gastos/{id}`:

```
private val remotos = FirebaseFirestore.getInstance()
    .collection("usuarios").document(uid).collection("gastos")

fun agregar(gasto: Gasto) {
    guardar(listOf(gasto) + gastos())                          // copia local
    remotos.document(gasto.id.toString()).set(aDocumento(gasto)) // nube
}

fun sincronizar(listo: () -> Unit) {
    remotos.get().addOnSuccessListener { resultado ->
        if (resultado.isEmpty) gastos().forEach { remotos.document(it.id.toString()).set(aDocumento(it))
}
        else guardar(resultado.documents.mapNotNull { desdeDocumento(it.id, it.data) })
        listo()
    }
}
```

`MainActivity` llama a `sincronizar` al abrir: en un teléfono nuevo aparecen los gastos de la cuenta, y la primera vez se suben los que ya estaban cargados. El importe se guarda como texto también en Firestore, porque un número ahí sería de punto flotante y perdería centavos. Si no hay conexión, el SDK encola la escritura y la manda solo cuando vuelve la red.

Las reglas de seguridad (`firestore.rules`) restringen cada colección a su dueño:

```
match /usuarios/{uid}/gastos/{gasto} {
  allow read, write: if request.auth != null && request.auth.uid == uid;
}
```

3.3 Firebase Cloud Messaging — `Avisos.kt` y `MainActivity.kt`

```
// MainActivity: al abrir, el teléfono se suscribe al tema y pide permiso para avisar
FirebaseMessaging.getInstance().subscribeToTopic("recordatorio-cierre")

// Avisos: con la app abierta el mensaje llega acá y hay que mostrarlo a mano;
// con la app cerrada lo muestra el sistema solo.
class Avisos : FirebaseMessagingService() {
    override fun onMessageReceived(mensaje: RemoteMessage) {
        val aviso = mensaje.notification ?: return
        mostrar(this, aviso.title.orEmpty(), aviso.body.orEmpty())
    }
}
```

El aviso se publica en el tema `recordatorio-cierre` desde la consola de Firebase (o desde un servidor con la API HTTP v1 de FCM) y le llega a cada teléfono suscripto.

3.4 DolarApi — `Cotizacion.kt` y `GastoEditorActivity.kt`

```
fun dolarTarjeta(): BigDecimal {
    val conexion = URL("https://dolarapi.com/v1/dolares/tarjeta").openConnection() as HttpURLConnection
    conexion.connectTimeout = 5_000
    conexion.readTimeout = 5_000
    try {
        if (conexion.responseCode != 200) error("La cotización respondió ${conexion.responseCode}")
        return BigDecimal(JSONObject(conexion.inputStream.bufferedReader().use { it.readText()
}).getString("venta"))
    } finally { conexion.disconnect() }
}
```

En la pantalla de carga se agrega la casilla «Importe en dólares». Si está marcada, al guardar se consulta la cotización en un hilo aparte (Android no permite red en el hilo principal), se convierte el importe a pesos y el comercio queda anotado con el valor usado, por ejemplo «Netflix (US$ 20 a 2028)». Si no hay conexión, el gasto no se guarda y la app lo dice, en lugar de guardar una conversión inventada.

4. Pruebas

Prueba	Resultado
Compilación del proyecto con las tres integraciones (`assembleDebug`)	Compila.
Pruebas unitarias del dominio y de la conversión (`testDebugUnitTest`)	Pasan. `CotizacionTest` : 20 US$ a 2028 = 40.560,00 pesos, sin perder centavos.
Llamada real a `GET https://dolarapi.com/v1/dolares/tarjeta`	Responde `{"moneda":"USD","casa":"tarjeta","compra":1963,"venta":2028,...}` (26/10/2026).

5. Cómo se conecta cada integración

Integración	Protocolo	Formato	Patrón	Quién inicia
Authentication	HTTPS, API REST	JSON	Pedido-respuesta	La app
Cloud Firestore	gRPC sobre HTTP/2	Protocol Buffers	Stream bidireccional: la app escribe y puede escuchar cambios en tiempo real; sin conexión, las escrituras esperan en cola	La app abre el canal y lo mantiene

Integración	Protocolo	Formato	Patrón	Quién inicia
Cloud Messaging	Publicar: HTTPS (API HTTP v1). Entregar: conexión TCP persistente de los Servicios de Google Play	JSON	Publicador/suscriptor por temas (*push*)	El servidor de Google: la app no pregunta, recibe
DolarApi	HTTPS, API REST	JSON	Pedido-respuesta (`GET`)	La app

Lo que no se usa, y por qué. *SOAP* es el estándar anterior a REST, con mensajes en XML y un contrato WSDL; sigue vivo en servicios estatales y bancarios, pero ninguno de los servicios que necesita Tarjetazo lo ofrece y en un teléfono es más pesado de procesar. Un *webhook* es una llamada HTTP de un servidor a otro cuando ocurre algo: el teléfono no puede recibirla, porque no tiene una dirección pública fija ni está siempre encendido. Si en el futuro un banco o una billetera avisara cada consumo por webhook, lo recibiría un servidor propio, que se lo reenviaría al teléfono por Cloud Messaging. *AMQP* (RabbitMQ y similares) es publicador/suscriptor entre servidores, con conexiones largas y pesadas para una batería; en el teléfono ese papel lo cumple Cloud Messaging, que comparte una sola conexión para todas las apps del dispositivo.

6. Arquitectura actualizada

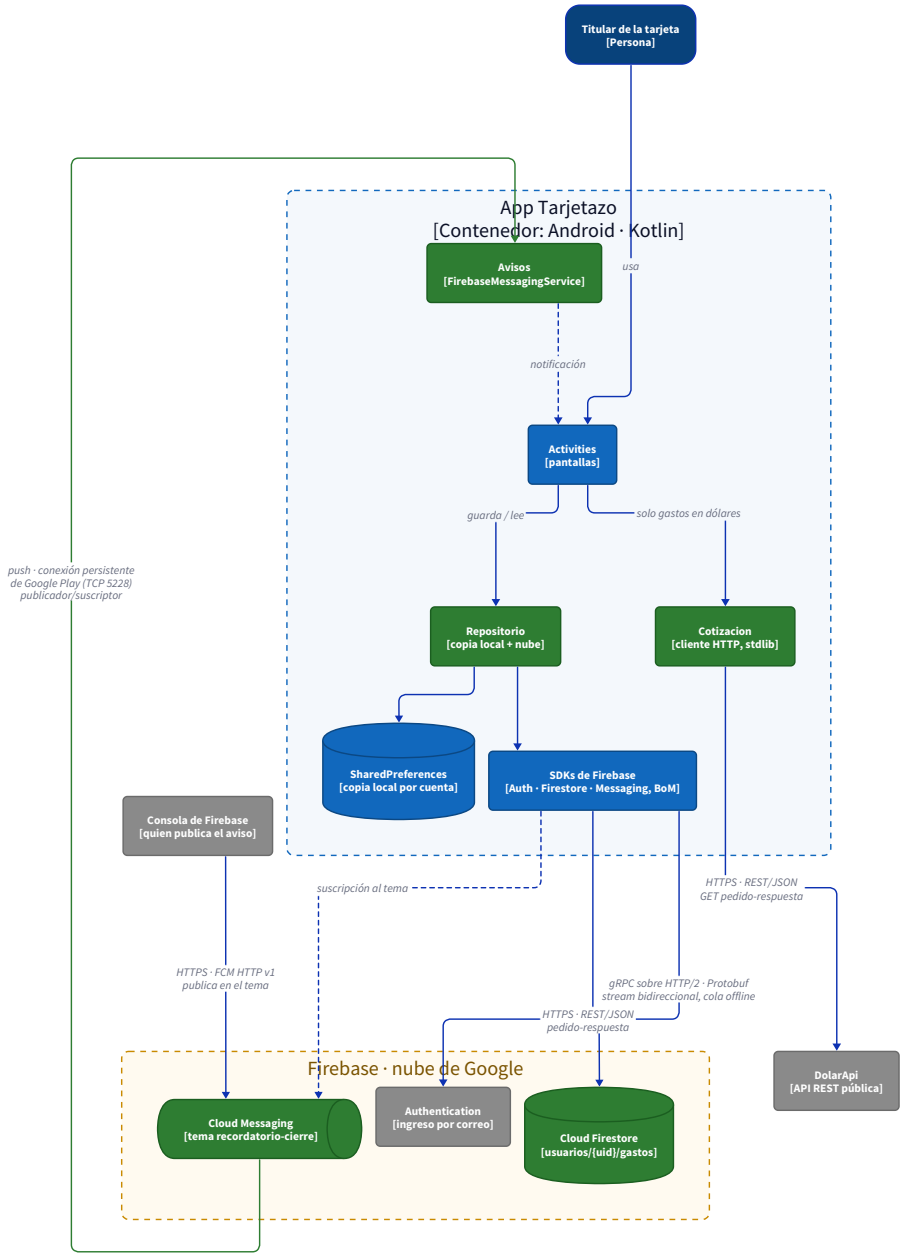


Diagrama de contenedores con la notación del modelo C4: en azul lo que ya existía, en verde lo agregado en este trabajo y en gris los servicios de terceros. Cada flecha indica el protocolo y el patrón de comunicación. Cómo mejora la aplicación cada integración:

- **Cloud Firestore** convierte a Tarjetazo de una libreta en un teléfono en una cuenta: los gastos sobreviven al cambio de dispositivo y cada persona ve solo los suyos. Es además la base para la próxima funcionalidad, invitar colaboradores a la tarjeta, que es lo que la primera participante de la prueba llamó «centralizar los gastos».
- **Cloud Messaging** le da a la app un canal de ida hacia el usuario: hasta ahora solo funcionaba si él se acordaba de abrirla.
- **DolarApi** cubre los consumos en moneda extranjera, que en una tarjeta argentina son frecuentes (suscripciones, compras en el exterior), con el mismo valor que después cobra el banco.